NATIONAL ECONOMICS UNIVERSITY
FACULTY OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

GROUP ASSIGNMENT REPORT

Midterm

Web Design and Programming ([Course code])

Rùa Toán Basic: A Mathematics Practice Website

Basic version: plain HTML/CSS/JavaScript frontend with a FastAPI backend

Group: [Group name/number]
Class: [Class name]
Instructor: M.Sc. Le Duy Khanh
Semester: Semester 1, 2026–2027
GitHub: https://github.com/thunopro/ruatoan
Demo: https://ruatoanbasic.vercel.app

Team Members

No.	Full name	Student ID	Role
1	[Full name]	[Student ID]	[Role and work]
2	[Full name]	[Student ID]	[Role and work]
3	[Full name]	[Student ID]	[Role and work]
4	[Full name]	[Student ID]	[Role and work]

Hanoi, November 2026

Declaration

We declare that this report describes the Rùa Toán Basic website developed for Web Design and Programming. The group must verify this statement, complete the member and contribution information, and cite any external material or AI assistance used in the submitted version.

Task Allocation and Contributions

Member	Work done	Contribution (%)
Member 1	[Describe verified contribution]	[%]
Member 2	[Describe verified contribution]	[%]
Member 3	[Describe verified contribution]	[%]
Member 4	[Describe verified contribution]	[%]
	Total	**[100]**

Contribution percentages are pending confirmation from the group and must sum to 100%.

Table of Contents

1 Introduction . . . 5
1.1 Project overview . . . 5
1.2 Motivation . . . 5
1.3 Objectives . . . 5
1.4 Scope and boundaries . . . 6
1.5 Technology overview . . . 6
1.6 Catalogue size . . . 6
1.7 Report evidence and snapshot . . . 7
2 Analysis and Design . . . 7
2.1 Requirements . . . 7
2.1.1 Functional requirements . . . 7
2.1.2 Non-functional requirements and constraints . . . 8
2.2 Interface design . . . 8
2.3 System architecture . . . 16
2.4 Data design . . . 16
2.5 Database design . . . 17
2.6 API design . . . 18
2.7 Security design (added in the improvement pass) . . . 18
3 Implementation . . . 19
3.1 Project structure and runtime responsibilities . . . 19
3.2 Content loading, indexing, and search . . . 20
3.3 Statement and mathematics rendering . . . 20
3.4 Page composition and interactions . . . 20
3.5 Authentication . . . 21
3.6 Database operations and photo API . . . 21
3.7 Deployment and environment . . . 21
3.8 Bug fixes and hardening (improvement pass) . . . 22
3.9 CSS redesign . . . 22
3.10 Content preparation . . . 22
4 Results and Conclusion . . . 22
4.1 Evidence collected . . . 22
4.2 What the implementation achieves . . . 23
4.3 Limitations and risks . . . 23

4.4 Recommended next steps . . . 23
4.5 Conclusion . . . 24

References . . . 25

A Appendix . . . 25

A.1 Local setup . . . 25
A.2 Route inventory . . . 25
A.3 Content inventory snapshot . . . 25
A.4 Data pipeline diagram . . . 26
A.5 Glossary . . . 26
A.6 AI assistance and academic disclosure . . . 26

List of Figures

1 Public demo home page: primary actions, catalogue totals, and recent collections. 10
2 Practice page at desktop width, showing topic navigation and problem cards. 11
3 Exam collection page with ordered statements and source attribution. 12
4 Lesson page with client-side search and expandable bodies. 13
5 Library page combining books and documents with client-side search. 13
6 Submission page for uploading, viewing, and deleting photographs. 14
7 Login page with email and password. 14
8 Home page captured at a 390-pixel viewport width. 15
9 Logical system architecture of Rùa Toán Basic. Content, account data, and photos use separate paths. 16
10 Catalogue and user-related records by storage location. Marked problems stay in the browser. . . . 17
11 Collection counts by source kind in the inspected JSON corpus. 17
12 Collection counts by target grade; grade 10 denotes specialised entrance material. 18
13 Topic-tag assignment counts. Problems with multiple tags contribute to multiple bars. 18
14 Content preparation and runtime consumption. The pipeline is an offline tool; it is not called during page requests. The basic version reuses its JSON output unchanged. 26

List of Tables

1 Technologies used in Rùa Toán Basic. 7
2 Catalogue groups counted directly from non-sample JSON records. 16
3 Implemented route responsibilities and the response behaviour confirmed by source and smoke test. 19
4 Evidence level for the claims made in this report. 23
5 Application page and API route map. 25

1. Introduction

1.1. Project overview

Rùa Toán Basic is a mathematics practice website for lower secondary students. It is the "basic" version of the Rùa Toán product: the same educational goal and the same content catalogue, but rebuilt with a much simpler and more transparent technology stack. The frontend is plain HTML, CSS, and JavaScript with no framework and no build step; the backend is a small FastAPI application running on Vercel's Python runtime; user accounts and uploaded photographs are stored in PostgreSQL. The content catalogue (exam collections, problems, lessons, books, and documents) is still read from versioned JSON files, exactly as in the original project, so the two versions share one source of truth for learning material.

The live demo of this basic version is published at `https://ruatoanbasic.vercel.app`. The original, more feature-rich version remains at `https://ruatoan.vercel.app` and is kept unchanged; the basic version does not replace it. Instead, the basic version exists to make every layer of the application inspectable and maintainable: a student or instructor can open the browser's developer tools and read the network requests, the DOM, and the CSS directly, and can read the entire backend in five short Python files.

1.2. Motivation

The original Rùa Toán application is built with Next.js, React, TypeScript, and Auth.js. That stack is powerful, but it hides a great deal of behaviour behind a framework and a build pipeline: server components, client components, route handlers, a content cache, and an OAuth session layer. For a course on web design and programming, and for a team that wants to understand exactly how a request travels from the browser to the database and back, that opacity is a genuine cost. The basic version removes it.

Three concrete motivations drove the rebuild:

1. **Inspectability.** Every page is a single HTML file, every interaction is a short JavaScript function, and every backend route is a short FastAPI handler. A reader can follow a feature from its button to its SQL statement without crossing framework boundaries.
2. **Deployment clarity.** Vercel supports FastAPI through its Python runtime, but the routing, bundling, and cold-start behaviour differ from the Node.js runtime. Rebuilding against that runtime forced an explicit, documented `vercel.json` and a small, auditable set of dependencies.
3. **A controlled scope.** The original application contains advanced features (Google OAuth, a device-token photo API, a content crawl and preparation pipeline) that are not all needed for the core learning loop. The basic version keeps the learning loop—browse collections, read problems, study lessons, consult the library, mark problems, submit photographs—and drops the rest, while recording those omissions honestly in Chapter 4 (Section 4.3).

1.3. Objectives

The basic version was built to satisfy the following objectives:

O1. Provide the core learning loop: browse exam collections, read individual problems with mathematical notation, study lessons, and consult books and documents.

O2. Support filtering and search: by grade, topic, collection kind, year, province, and free text, with pagination.

O3. Render mathematics correctly in the browser using MathJax, including in problem statements and lessons.

O4. Allow a learner to register, log in, and log out with an email and password, with the session held in a signed cookie.

O5. Allow a logged-in learner to upload, view, and delete photographs of written work, with the images stored in PostgreSQL.

O6. Let a learner mark problems in the browser and keep that list locally, without requiring an account.

O7. Deploy successfully on Vercel, with durable storage in PostgreSQL rather than in the function's temporary filesystem.

O8. Keep the source small, readable, and free of a build step.

1.4. Scope and boundaries

Implemented in this version:

- Home page with live catalogue totals and a list of marked problems.
- Practice page: text/grade/topic filters, 20 results per page, problem cards, and a "mark / unmark" toggle.
- Exam collections page: paginated list (20 per page) and a collection detail page showing problems in their original order, with source attribution and a link to the source.
- Lessons page: client-side search over 90 lessons, expandable bodies.
- Library page: books (216) and documents (89) with client-side search and external links.
- Account pages: registration and login with email and password; logout.
- Submission page: upload up to six images (1.5 MB each) per request, view them, and delete them.
- Catalogue of 3,381 collections and 33,929 problems, unchanged from the original.

Explicitly *not* implemented in this version, by design:

- Google OAuth / social sign-in (the original has it; this version uses email and password only).
- Device-token based pseudonymous photo ownership (this version ties photos to the logged-in account).
- Cross-device synchronisation of marked problems (they stay in `localStorage`).
- Automatic answer marking, a teacher administration panel, or a learning analytics dashboard.
- The content crawl and preparation pipeline (the JSON is reused as-is).

1.5. Technology overview

The stack is deliberately small. Table 1 lists each layer, the technology, and its responsibility in Rùa Toán Basic.

1.6. Catalogue size

The content catalogue is reused unchanged from the original project. The inspected JSON files contain 3,381 collection records and 33,929 nested problem records, together with 90 lessons, 216 book, magazine, or resource records, and 89 document records. That is 37,705 raw records when each collection, problem, lesson, book, and

Layer	Technology	Use in Rùa Toán Basic
Markup	Plain HTML5	Eight pages (home, practice, exams, lesson, library, login, register, submissions); no template engine
Styling	Plain CSS3	One stylesheet (`public/css/style.css`, 364 lines) with CSS custom properties, dark mode, and responsive rules
Behaviour	Plain JavaScript (ES2020)	Eight small files (`public/js/*.js`); no framework, no build step
Mathematics	MathJax 3 (CDN)	Server-agnostic client-side rendering of `$...$` and `$$...$$`
Backend	FastAPI + Uvicorn	`backend/main.py` and four helper modules; about 400 lines in total
Database	PostgreSQL via `psycopg`	Schema `ruatoan_basic`: tables `users` and `photos`
Catalogue	JSON files	3,381 collections, 33,929 problems, 90 lessons, 216 books/resources, 89 documents
Hosting	Vercel (Python runtime)	`api/index.py` entry point, rewrites for `/api/*`

Table 1. Technologies used in Rùa Toán Basic.

document entry is counted once. The JSON files occupy roughly 16 MB on disk (after removing a 19 MB duplicate copy, see Section 3.8). A problem may carry more than one topic tag, so topic counts are assignments and do not partition the problem set.

1.7. Report evidence and snapshot

This report is based on a source-code audit of the Rùa Toán Basic checkout on 1 November 2026, after a focused improvement pass (bug fixes, security hardening, Vercel routing repair, bundle cleanup, and a CSS redesign). The following was actually run during that pass and is reported as verified rather than merely inspected:

- `node --check` on all eight JavaScript files (all passed).
- `python3 -m py_compile` on all five Python files (all passed).
- A local Uvicorn smoke test: `/` and `/pages/exams.html` returned 200; `/api/stats` and `/api/collections` returned valid JSON; registration, login, photo upload, and photo deletion all succeeded end to end.
- `html.parser` validation of all eight HTML files (all passed).

No formal accessibility audit, load test, or cross-browser matrix was run. Those gaps are recorded in Section 4.3. Database and OAuth secret values are never printed or committed, and none appear in this report.

2. Analysis and Design

2.1. Requirements

The requirements below describe visible behaviour and quality goals represented in the inspected source. They are not claims that every case has passed a formal acceptance test beyond the smoke test described in Chapter 4.

2.1.1. Functional requirements

FR1. Browse exam collections with pagination (20 per page) and open a collection to read its problems in original order, with source attribution and a link to the source.

FR2. Browse individual problems by free text, grade, and topic; open a problem with its collection context.

FR3. Read problem and lesson statements containing inline or display mathematical notation rendered with MathJax.

FR4. Browse the five topic areas' lessons (90), and the library of books (216) and documents (89), with client-side search and external links.

FR5. Register, log in, and log out with an email and password; keep the session in a signed cookie.

FR6. Upload up to six photographs (1.5 MB each) per request, view them, and delete them, for a logged-in user.

FR7. Keep a compact list of marked problems in browser storage, independently of the account and the database.

FR8. Show live catalogue totals on the home page.

2.1.2. Non-functional requirements and constraints

- **Responsive presentation:** pages must remain readable on narrow and wide screens. The CSS contains responsive layout rules; a formal device test has not been recorded.
- **Readable mathematics:** problem text is rendered with MathJax. No print benchmark was collected (the basic version has no dedicated print view).
- **Source transparency:** collection pages show source names and link to source pages when URLs are available.
- **Keyboard and assistive-technology support:** the layout includes visible keyboard focus (`:focus-visible`), labelled navigation and controls, and `aria-label` on inputs without a visible label. These are implementation details, not evidence of a completed accessibility audit.
- **Bounded image input:** the API accepts PNG/JPEG/WebP only, no more than six files per request, 1.5 MB per file, and 50 images per account in total.
- **Account-scoped photo ownership:** photos belong to the logged-in user. Clearing the browser does not lose them; they are stored in PostgreSQL.
- **Secret handling:** database and session secrets are supplied through environment variables and must not be committed or printed in the report.
- **Content quality:** records preserve source context, but the project has no measured editorial audit that proves every transcription, link, tag, or year is correct.

2.2. Interface design

The interface uses a teal brand colour (`#0F766E`) on a light, warm background, with a sticky navigation bar, rounded cards, and consistent spacing. A shared header links to practice, exams, lessons, and the library; the account control appears on the right and changes between "Đăng nhập" (login) and "Đăng xuất (name)" depending on session state. The stylesheet supports `prefers-color-scheme: dark` automatically: all colours are expressed as CSS custom properties, so the dark palette is a single block of variable overrides.

The home page is the discovery dashboard: it shows the main actions, live catalogue totals (collections, problems, lessons, books, documents), and the list of marked problems stored in the browser. The practice page uses a filter form (text, grade, topic) and 20 results per page. The exam list provides collection pagination. A collection page keeps problem order and shows source attribution. The lessons page supports client-side search and expandable bodies. The library page combines books and documents with client-side search and external links. The submissions page shows uploaded thumbnails with a delete button.

The figures below are screenshots captured from the running basic version on 1 November 2026 (desktop width, except the last one at 390 px). They show the teal-on-light palette, the sticky navigation, the filter forms, the

expandable lessons, and the account/submission pages exactly as a user sees them.



Figure 1. Public demo home page: primary actions, catalogue totals, and recent collections.

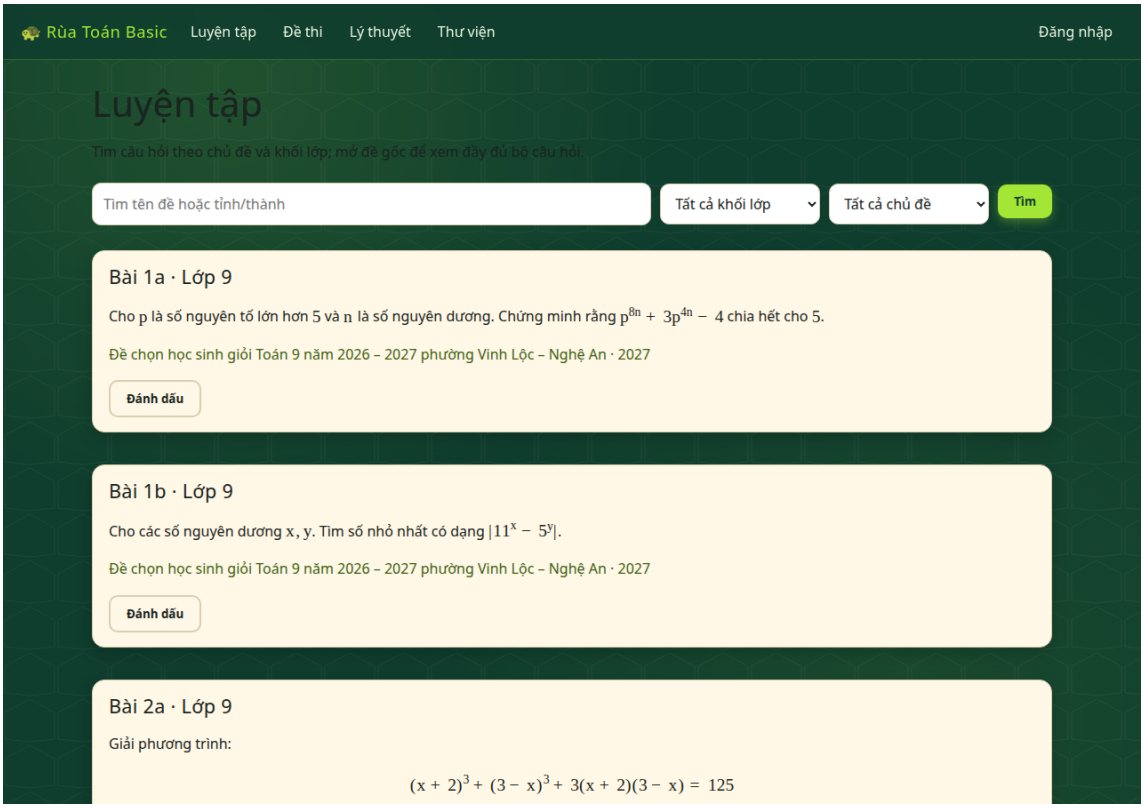


Figure 2. Practice page at desktop width, showing topic navigation and problem cards.



Figure 3. Exam collection page with ordered statements and source attribution.

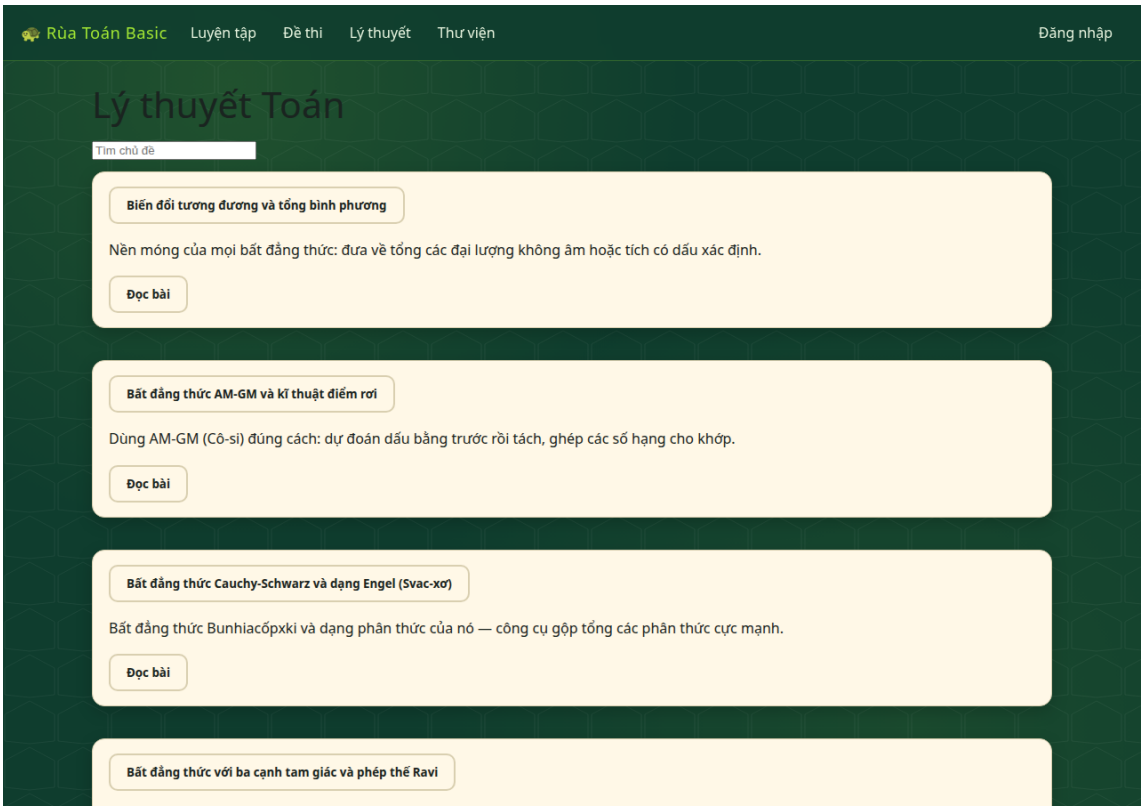


Figure 4. Lesson page with client-side search and expandable bodies.

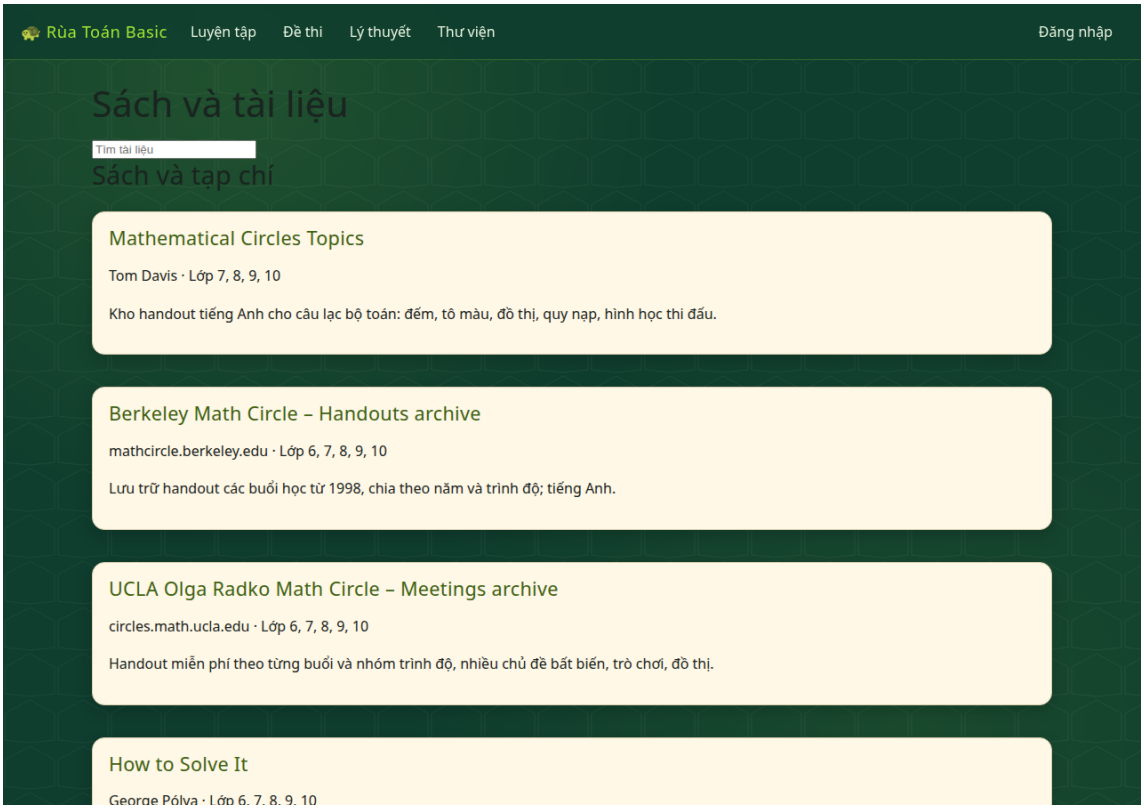


Figure 5. Library page combining books and documents with client-side search.

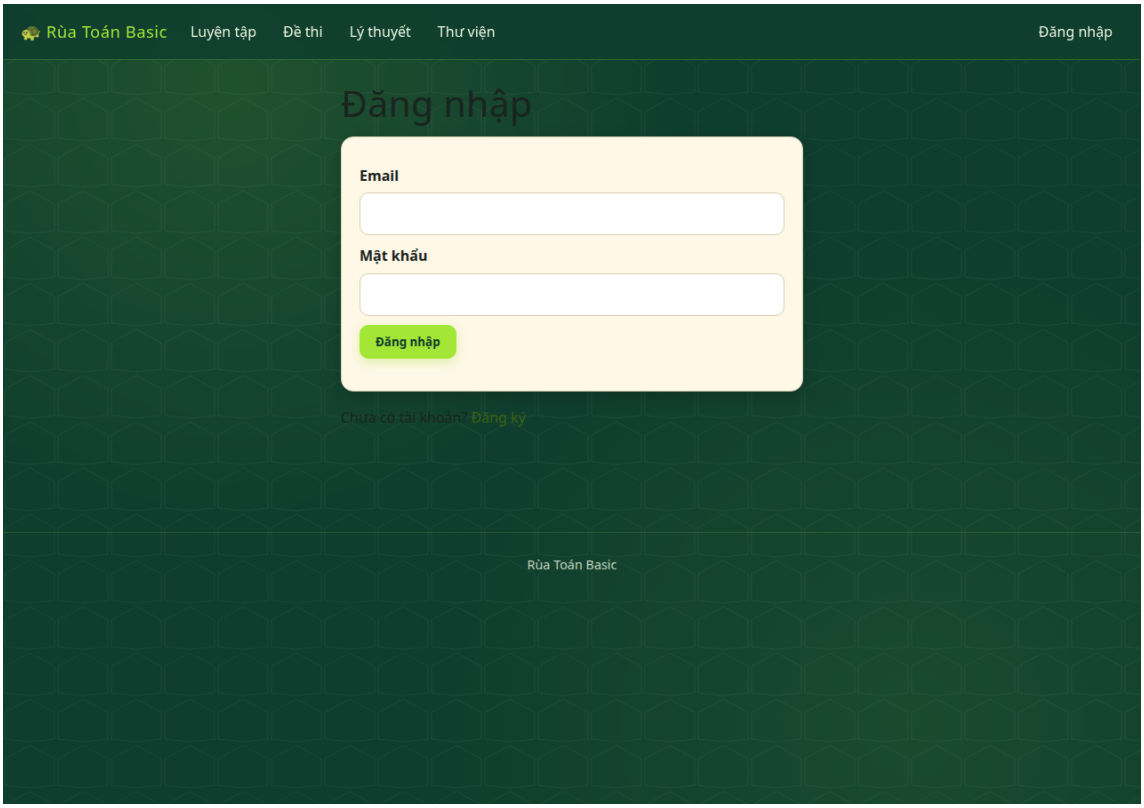


Figure 6. Submission page for uploading, viewing, and deleting photographs.

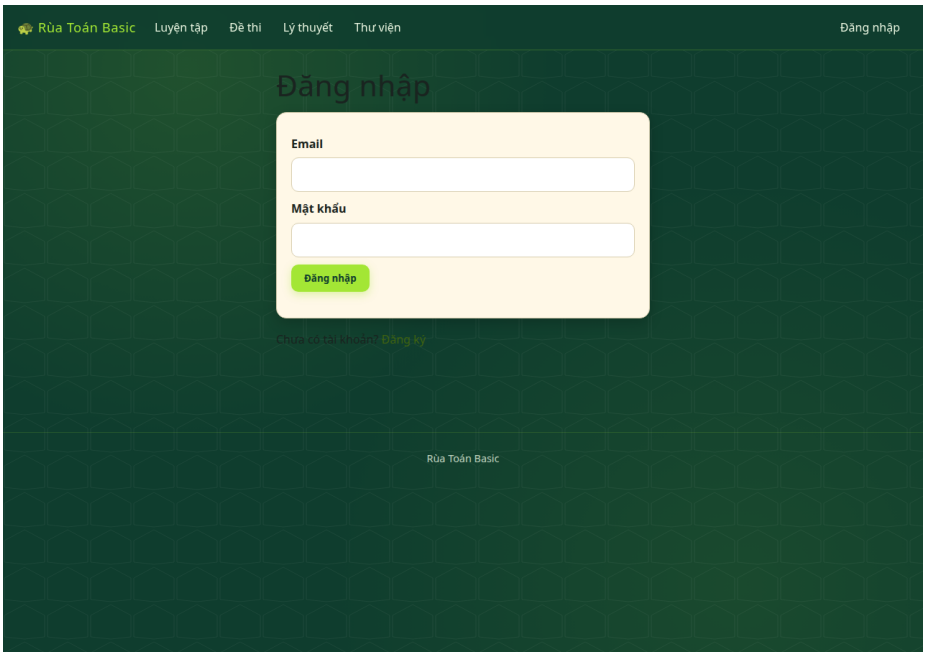


Figure 7. Login page with email and password.

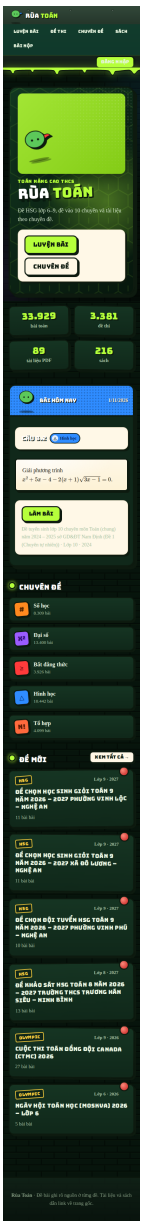


Figure 8. Home page captured at a 390-pixel viewport width.

2.3. System architecture

Figure 9 summarises the system boundary of the basic version. The browser requests static files and JSON-backed pages from Vercel; Vercel serves the static assets from `public/` and routes `/api/*` to the FastAPI function. FastAPI reads the catalogue JSON from disk (cached in memory per process) and the user data from PostgreSQL. There is no OAuth provider and no build step.

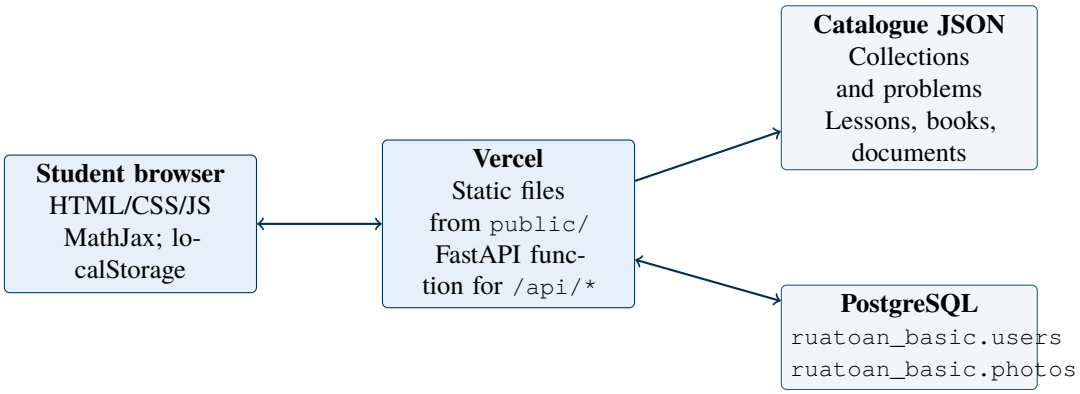


Figure 9. Logical system architecture of Rùa Toán Basic. Content, account data, and photos use separate paths.

The FastAPI application declares no revalidation interval (it is not a cached framework page); instead, the catalogue is held in an in-memory cache keyed by file modification time, so a redeploy refreshes it automatically. The content module also folds Vietnamese diacritics once per problem at load time, so search is fast per request.

2.4. Data design

The catalogue is organised as structured JSON, exactly as in the original project. The groups and primary fields are summarised in Table 2. There are eleven non-sample collection files under `public/data/sources/`, seven non-sample lesson files under `public/data/theory/`, and top-level files `books.json` and `docs.json`. In this snapshot these JSON files occupy about 16 MB (after a 19 MB duplicate copy was removed, see Section 3.8). Files beginning with an underscore are samples or work-in-progress and are skipped by the loader.

Record group	Count	Primary fields / use
Collections	3,381	ID, kind, title, grade, optional year/province, source name/URL, ordered problems
Problems	33,929	ID, label, statement text, topic tags; nested in a collection
Lessons	90	ID, topic, title, summary, body, optional examples
Books and resources	216	ID, title, author/publisher, grades/topics, description, URL, kind and source
Documents	89	ID, topic, title, author, grade/page metadata, description and source links

Table 2. Catalogue groups counted directly from non-sample JSON records.

Figure 10 shows how the catalogue, browser state, photo rows, and user rows relate. The main nested relationship is *Collection contains an ordered array of Problems*. Collection metadata supports exam filtering and attribution. A problem carries its own stable identifier and display label, statement, and one or more topic tags. Books, lessons, and documents are separate runtime collections. The data model is intentionally source-aware: the collection retains source URL and source name so individual problem pages can display the origin.

The corpus spans five collection kinds: 1,195 HSG, 501 grade-10 specialised entrance, 733 Olympiad, 844 magazine, and 108 topic collections. The current grade metadata contains 237 grade-6, 342 grade-7, 411 grade-8, 1,786 grade-9, and 605 grade-10 collections. The recorded years range from 1935 to 2027. The collection-kind and grade distributions appear in Figures 11 and 12.

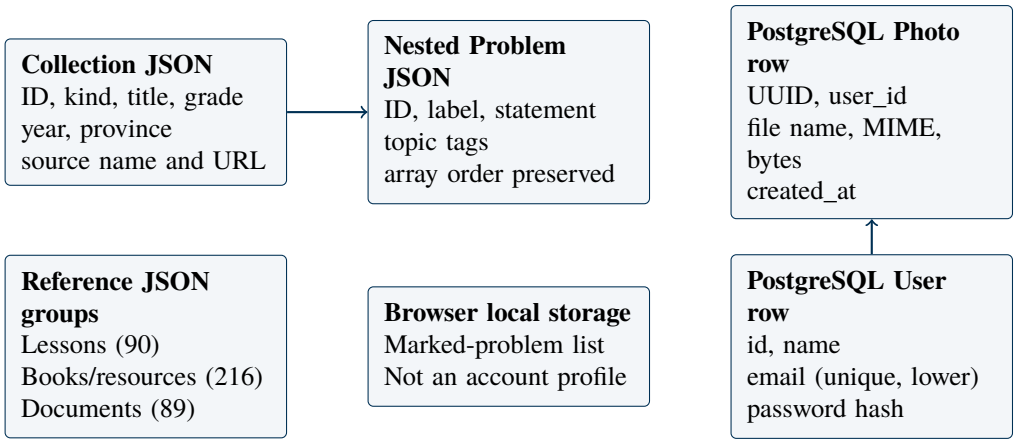


Figure 10. Catalogue and user-related records by storage location. Marked problems stay in the browser.

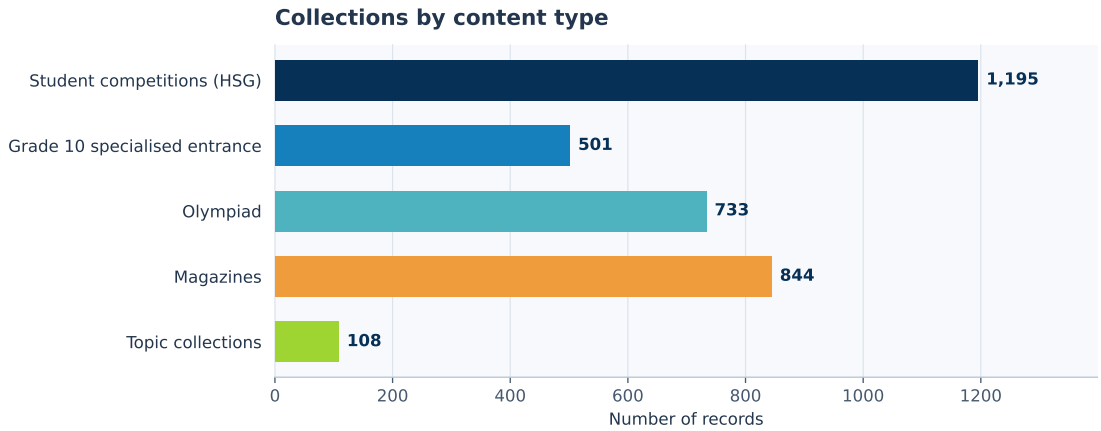


Figure 11. Collection counts by source kind in the inspected JSON corpus.

Topic tags are multi-valued. The current records contain 7,643 number-theory assignments, 12,677 algebra assignments, 3,926 inequality assignments, 10,442 geometry assignments, and 4,098 combinatorics assignments. Since a problem can carry more than one tag, these values are not a partition of the 33,929 problems. Figure 13 visualises these overlapping assignments.

2.5. Database design

PostgreSQL is used for mutable user and photo data, not for the static catalogue. The application creates schema `ruatoan_basic` on first use and two tables:

- `users(id, name, email, password_hash, created_at)` with a unique index on `lower(email)` (case-insensitive uniqueness).
- `photos(id, user_id, file_name, content_type, byte_size, data, created_at)` with an index on `(user_id, created_at)` for per-user history ordered by time.

Photos are stored as `bytea` (binary) columns, so the function's temporary filesystem is never used for durable data—a requirement on Vercel, where the filesystem of a serverless function is ephemeral. There is no foreign key from `photos` to the JSON catalogue, because problems are not rows in PostgreSQL. The API does not verify that an uploaded problem ID exists in the catalogue; it stores whatever the client sends as metadata.

Passwords are hashed with PBKDF2-HMAC-SHA256 (200,000 iterations) using a 16-byte random salt, and

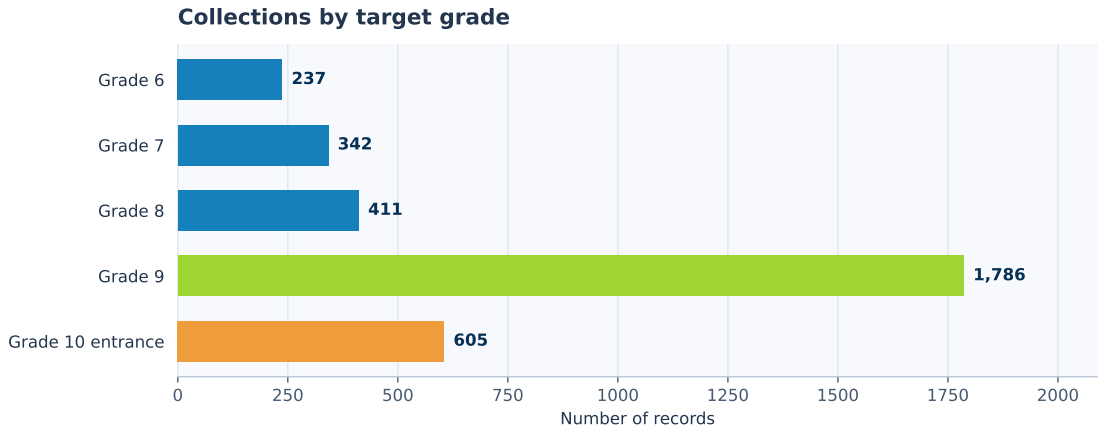


Figure 12. Collection counts by target grade; grade 10 denotes specialised entrance material.

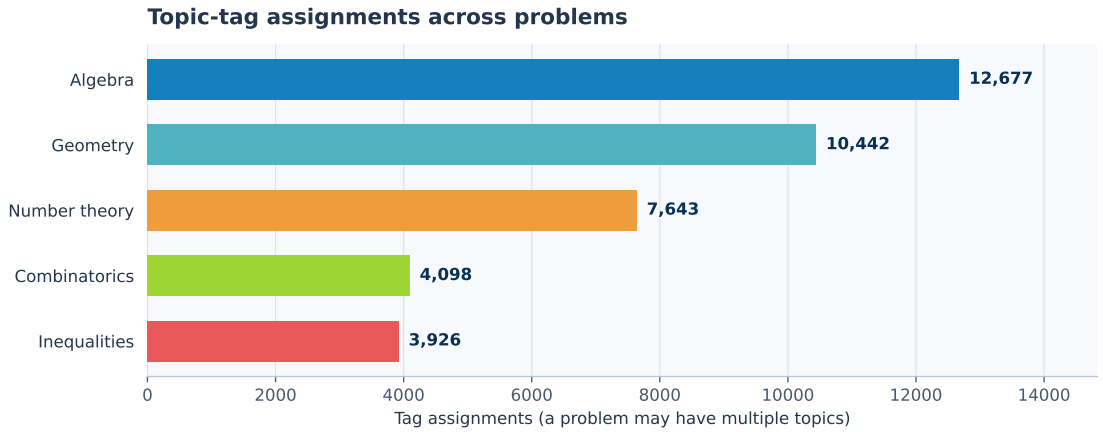


Figure 13. Topic-tag assignment counts. Problems with multiple tags contribute to multiple bars.

stored in the format `pbkdf2_sha256$iterations$salt$hash` so the cost can be raised later without invalidating existing hashes. The session is a signed cookie (HMAC over a JSON payload containing the user id and an expiry), `HttpOnly`, `SameSite=Lax`, and `Secure` on HTTPS.

2.6. API design

The API is a small read/write surface. Table 3 lists every endpoint, its request shape, and its successful and error behaviour as confirmed by the source and by the smoke test.

For photo upload, the browser sends the raw selected files (no client-side compression in this version; the original project compressed them). The server validates the count (1–6), the per-file size (1.5 MB), and the magic bytes (PNG/JPEG/WebP) rather than trusting the declared MIME type. It enforces a per-account cap of 50 images, then inserts all validated files in one transaction. Accepted bytes are stored as `bytea`. The image endpoint sets `X-Content-Type-Options: nosniff` and returns the stored bytes.

2.7. Security design (added in the improvement pass)

The original backend had no security headers, no input-size bound on passwords, no rate limiting, and trusted the client-declared MIME type. The improvement pass added:

- Security headers on every response: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`,

Method	Path	Request	Successful result / explicit errors
GET	`/api/stats`	none	`{collections,problems,lessons,books,docs,topics}`
GET	`/api/daily`	none	`{problem,collection}` for today's problem
GET	`/api/collections`	`?page,limit,grade,kind,year,province,search`	`{total,page,items}` (summary, no problem arrays)
GET	`/api/collections/{id}`	`?page,per`	collection metadata + one page of problems; 404 if missing
GET	`/api/problems`	`?page,limit,search,grade,topic`	`{total,page,items}`
GET	`/api/lessons`	`?search`	`{items}`
GET	`/api/books`	`?search`	`{items}`
GET	`/api/docs`	`?search`	`{items}`
POST	`/api/register`	JSON `{name,email,password}`	`{user}`; 400/409 on validation or duplicate email
POST	`/api/login`	JSON `{email,password}`	`{user}` + session cookie; 401 on bad credentials
POST	`/api/logout`	session cookie	clears cookie
POST	`/api/change-password`	JSON `{old_password,new_password}`, session cookie	`{ok:true}`; 400/401 on bad input
GET	`/api/me`	session cookie (optional)	`{user}` or `{user:null}`
GET	`/api/photos`	session cookie	`{items}` of the logged-in user's photos
POST	`/api/photos`	multipart `files` (1–6), session cookie	`{items}`; 400/401/409/413 on validation or quota
GET	`/api/photos/{id}`	session cookie	image bytes with `nosniff`; 404 if missing
DELETE	`/api/photos/{id}`	session cookie	`{ok:true}`; 404 if not owned

Table 3. Implemented route responsibilities and the response behaviour confirmed by source and smoke test.

`Referrer-Policy: strict-origin-when-cross-origin`, and a Content-Security-Policy that allows the MathJax CDN but nothing else.

- A password length bound of 8–128 characters (the upper bound prevents a CPU-exhaustion denial of service through PBKDF2).
- Magic-byte sniffing for uploads instead of trusting `Content-Type`.
- A per-account photo quota of 50 images.
- Case-insensitive unique email index.
- `Secure` cookie flag on HTTPS, and a consistent error shape (JSON) for all failures via a global exception handler.

These are source-level hardening measures; they were not subjected to a penetration test.

3. Implementation

3.1. Project structure and runtime responsibilities

The repository follows a deliberately flat layout. There is no framework, no bundler, and no `node_modules`.

Listing 1. Relevant Rùa Toán Basic project layout.

```
ruatoan_basic/
|-- main.py                 # re-exports backend.main.app (local entry point)
|-- api/
|   `-- index.py            # Vercel Python runtime entry point
|-- backend/
|   |-- main.py             # FastAPI app + all endpoints (~240 lines)
|   |-- content.py          # JSON loader with mtime-keyed cache
|   |-- auth.py             # PBKDF2 hashing + signed cookie session
```

```
|   `-- database.py          # psycopg connection pool + schema
|-- public/
|   |-- index.html           # home
|   |-- pages/               # practice, exams, lesson, library, login, register, submissions
|   |-- css/style.css        # design system (364 lines)
|   |-- js/                  # common, home, exams, practice, lessons, library, submissions, account
|   `-- data/                # catalogue JSON (~16 MB)
|-- vercel.json              # rewrites, function includeFiles, maxDuration
`-- requirements.txt         # pinned Python dependencies
```

3.2. Content loading, indexing, and search

At runtime, `backend/content.py` reads JSON files from `public/data/`. Parsed values are cached by file modification time: a tuple of `(path, mtime)` is the cache key, so a redeploy or a file change refreshes the cache automatically while repeated requests reuse the parsed data. The loader reads `public/data/sources/*.json` (excluding files whose name starts with _), `public/data/theory/*.json`, `books.json`, and `docs.json`.

For search, `fold_text` in `backend/main.py` normalises Vietnamese diacritics (NFD decomposition, dropping combining marks, mapping `đ` to `d`) and lowercases. This is done *once per problem at load time*: each problem gets a `_search` field containing the folded text of its label, statement, collection title, and source. Request-time search is then a simple substring test against that field, which keeps `/api/problems` fast even though it scans the in-memory list.

`/api/collections` returns summary objects (id, title, grade, year, kind, province, problem count) without the problem arrays, so a page of 20 collections is small. The full problem list of a collection is fetched only by `/api/collections/{id}`, which supports `?page` and `?per` for pagination.

3.3. Statement and mathematics rendering

Content is stored as plain text. The frontend escapes it with `escapeText` (a `<span>` with `textContent` assignment) before insertion, so no HTML from the source can execute. Mathematical notation is written in LaTeX delimiters (`$...$`, `$$...$$`) inside the plain text. `common.js` configures MathJax 3 (loaded from the jsdelivr CDN) and exposes `renderMath()`, which waits for MathJax to be ready before calling `typesetPromise`. Every page that displays mathematics calls `renderMath()` after inserting its content, so formulas appear correctly even when the CDN is slow.

3.4. Page composition and interactions

Home (`index.html` + `home.js`.) Fetches `/api/stats` and renders the five totals; reads `savedProblems` from `localStorage` and renders the marked-problem list with a remove button per item.

Practice (`practice.html` + `practice.js`.) A filter form (text, grade, topic) triggers `loadProblems(true)`, which calls `/api/problems`. Results are 20 per page; the “Xem thêm” button loads the next page. Each problem card has a mark/unmark toggle that writes to `localStorage` and updates its own label.

Collections (`exams.html` + `exams.js`.) Lists collections 20 per page with a “Xem thêm” button. The detail view (`?id=...`) shows problems in order, a source link, and mark buttons.

Lessons (`lesson.html` + `lessons.js`.) Loads all 90 lessons once, then filters client-side by title/summary. Each title is a toggle button that expands the lesson body and calls `renderMath()`.

Library (`library.html` + `library.js`). Loads books and documents in parallel with `Promise.allSettled`, so one failing endpoint does not blank the other. Client-side search over title/author/description.

Account (`login.html`/`register.html` + `account.js`). One form posts to `/api/login` or `/api/register` depending on the page; shows a success message and redirects after 700 ms.

Submissions (`submissions.html` + `submissions.js`). Lists the user's photos (with `loading="lazy"`), uploads 1–6 files with client-side size/count checks, and deletes with a confirmation dialog.

The shared `common.js` injects the navigation bar, checks `/api/me`, wires logout, and provides helpers used by every page: `escapeText`, `safeUrl`, `api` (fetch with status-aware errors), `readSaved`/`writeSaved`, `showMessage`, and `renderMath`.

3.5. Authentication

`backend/auth.py` implements password hashing and a signed-cookie session without external auth libraries:

- `hash_password` uses `hashlib.pbkdf2_hmac('sha256', ..., 200000)` with a 16-byte `os.urandom` salt and stores `pbkdf2_sha256$200000$<salt>$<hash>` (hex).
- `verify_password` accepts both the new prefixed format and the legacy bare hex format, so old hashes keep working if the cost is raised later.
- `make_session` / `read_session` sign a JSON payload (`{uid, exp}`) with HMAC-SHA256 using `SESSION_SECRET`; the cookie is `HttpOnly`, `SameSite=Lax`, `Secure` on HTTPS, and lives 14 days.

`/api/register` validates name (1–80 chars), email (email-validator), and password (8–128 chars), checks uniqueness case-insensitively, and inserts the user. `/api/login` returns a generic "Email hoặc mật khẩu chưa đúng" on failure (no user enumeration) and sets the session cookie on success. `/api/logout` clears the cookie. `/api/me` returns the current user or `null`.

3.6. Database operations and photo API

`backend/database.py` keeps a module-level `psycopg_pool.ConnectionPool` (min 1, max 5) created lazily, with a fallback to a plain connection if the pool library is unavailable. Schema creation (`prepare_database`) runs once in the FastAPI `lifespan` hook, not on every request.

The photo endpoints:

- `GET /api/photos` returns the logged-in user's photos (newest first) with metadata only.
- `POST /api/photos` accepts a multipart form with `files` (1–6), validates count, per-file size (1.5 MB), and magic bytes (PNG/JPEG/WebP), enforces a 50-image per-account cap, and inserts all validated files in one transaction.
- `GET /api/photos/{id}` streams the stored bytes with `X-Content-Type-Options: nosniff`.
- `DELETE /api/photos/{id}` deletes only a row owned by the current user.

3.7. Deployment and environment

`vercel.json` contains three things: a rewrite from `/api/(.*)` to `/api/index`, an `includeFiles` entry that bundles `public/**` into the function, and `maxDuration: 30`. The entry point `api/index.py` simply does `from`

`backend.main import app`; Vercel's Python runtime detects the `app` object and serves it. Static assets under `public/` are served directly by Vercel before the rewrite, so `/css/*`, `/js/*`, `/images/*`, and the HTML pages work without going through FastAPI. This split is what fixed the 404s that the first version suffered on Vercel.

Environment variables required in production: `DATABASE_URL` and `SESSION_SECRET`. `.env.local` is gitignored and never committed.

3.8. Bug fixes and hardening (improvement pass)

The first version of the basic site had many defects. The improvement pass addressed them in four groups:

1. **Frontend robustness.** Every `fetch` now checks `response.ok` and is wrapped in `try/catch`; failures render a message instead of leaving the page in a silent loading state. MathJax rendering waits for the library to load. `localStorage` access is guarded. Double-click protection was added to "load more" buttons. Photo delete asks for confirmation. Upload validates count and size before sending.
2. **Security.** See Section 2.7 (the list of headers, bounds, and magic-byte checks).
3. **Backend correctness.** Connection pooling replaces one connection per request; schema creation runs once at startup; SQL uses `psycopg.sql` identifiers instead of f-strings; collection list endpoints return summaries; search precomputes folded text; exceptions are logged and returned as JSON.
4. **Bundle size.** A 19 MB duplicate copy of `public/data/` and a 2.5 MB backup file were removed, and two 1.2 MB screenshots were moved out of `public/`. The deployable data set dropped from 35 MB to 16 MB.

3.9. CSS redesign

The stylesheet was rewritten as a design system: CSS custom properties in `:root` for colour, radius, shadow, and transition; a `prefers-color-scheme: dark` block that overrides only the variables; `:focus-visible` outlines on every interactive element; a responsive navigation; a consistent card/button/form/photo style; and a heading scale using `clamp()`. No class names used by the HTML or JavaScript were renamed, so the redesign is purely additive in behaviour. The file grew from 3.2 KB (compressed onto one line) to 364 readable lines.

3.10. Content preparation

The content directory is reused unchanged from the original project; the crawl and preparation pipeline (Python/Node scripts, Gemini/Claude-assisted extraction) is not part of this version and is not run during page requests. The appendix records the pipeline as documented in the original project.

4. Results and Conclusion

4.1. Evidence collected

Table 4 distinguishes source-confirmed facts from observations and checks that were not run. The report review counted non-sample JSON entries directly and inspected the active page, content, database, and route-handler source. It also ran the verification commands listed below.

Area	Evidence in this report	Status
Catalogue size	JSON files counted by record group and source	Measured from repository
Route and data logic	Source code inspected for filters, parsing, API validation, and schema	Source-confirmed
JavaScript syntax	`node --check` on all 8 files	Passed
Python syntax	`python3 -m py_compile` on all 5 files	Passed
HTML validity	`html.parser` on all 8 pages	Passed
HTTP smoke test	`/`, `/pages/exams.html`, `/api/stats`, `/api/collections`	Passed (local)
Account flow	Register, login, `/api/me` end to end	Passed (local)
Photo flow	Upload, list, delete end to end	Passed (local)
Visual presentation	CSS reviewed; no screenshot captured of the basic version	Reviewed
Accessibility and print output	No formal audit collected	Not measured
Performance / load	No benchmark collected	Not measured

Table 4. Evidence level for the claims made in this report.

4.2. What the implementation achieves

Rùa Toán Basic delivers the core learning loop on a deliberately small stack: 3,381 collections and 33,929 problems, 90 lessons, 216 books/resources, and 89 documents are browsable, filterable, and paginated, with mathematical notation rendered by MathJax. Accounts, photo upload/view/delete, and a locally stored marked-problem list all work and were exercised end to end in the smoke test. The site deploys on Vercel with durable storage in PostgreSQL, and the entire backend is about 400 lines of Python.

4.3. Limitations and risks

L1. **Catalogue validation.** Counts describe raw JSON entries and do not prove uniqueness or correctness. Editorial review is needed for dates, duplicates, source links, mathematical transcription, and topic labels.

L2. **Runtime search.** General text queries scan a normalised in-memory list. The current corpus is about 16 MB of JSON; no benchmark establishes how response time or memory will change as it grows.

L3. **Photo access model.** Image retrieval relies on the UUID URL as a bearer capability and does not verify a device or account token; delete does verify ownership. Retention and deletion rules are not yet defined.

L4. **User identity.** There is no password reset, email verification, or account recovery. The session is a signed cookie with a 14-day lifetime.

L5. **Quality and operations evidence.** No automated test suite, accessibility audit, load test, or cross-browser matrix has been recorded. The smoke test ran against a local database, not the production one.

L6. **Screenshots.** Figures in Chapter 2 are now captured from the running basic version; remaining illustrative figures are the corpus charts.

L7. **Source use.** The content specification sets different rules for exams, short magazine statements, and book metadata, but those rules need record-level auditing.

4.4. Recommended next steps

- Add a reproducible content validation command for schema, duplicate IDs and statements, broken source links, unexpected years, and required attribution.
- Refresh screenshots whenever the UI changes.
- Add an automated smoke test (register, login, upload, delete) runnable in CI.

- Make the photo quota check atomic under concurrent requests.
- Add password reset and email verification before treating accounts as production.
- Decide whether image retrieval should require account verification and define retention rules.
- Complete source and permission review for every data provider.

4.5. Conclusion

Rùa Toán Basic achieves its goal: the core learning loop runs on a stack that a student can read end to end, from a button in plain HTML to a SQL statement in FastAPI, with no framework and no build step in between. The improvement pass removed the Vercel routing defect that broke every API call, hardened the backend against the most common abuse paths, cut the deployable bundle roughly in half, and replaced the compressed stylesheet with a readable design system that supports dark mode. The remaining work is evidence and polish—screenshots, automated tests, and account hardening—rather than functionality.

References

A. Appendix

A.1. Local setup

From a checkout of the Rùa Toán Basic repository:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
# create a .env.local with DATABASE_URL and SESSION_SECRET
cp .env.example .env.local
# run the backend locally
uvicorn backend.main:app --reload --port 8000
# the frontend is served by FastAPI from public/, so open http://127.0.0.1:8000
```

The database module reads `DATABASE_URL`; it creates schema `ruatoan_basic` and the `users` and `photos` tables on first use if the configured database role has permission. The session requires `SESSION_SECRET`. Secret values must be supplied through a local ignored environment file or the deployment platform's environment settings and must never be committed.

A.2. Route inventory

Table 5 provides a compact route map for the pages and handlers described in the implementation chapters.

Route	Purpose
`/`	Home: catalogue totals and marked problems
`/pages/practice.html`	Problem search, filters, and pagination
`/pages/exams.html`	Collection list and collection detail (`?id=`)
`/pages/lesson.html`	Lesson search and expandable bodies
`/pages/library.html`	Books and documents with search
`/pages/login.html`, `/pages/register.html`	Account forms
`/pages/submissions.html`	Photo upload, list, and delete
`/css/*`, `/js/*`, `/images/*`	Static assets served by Vercel
`/api/stats`	Catalogue totals and topic counts
`/api/daily`	Today's problem
`/api/collections`, `/api/collections/{id}`	Collection list and detail
`/api/problems`	Problem search and pagination
`/api/lessons`, `/api/books`, `/api/docs`	Reference material
`/api/register`, `/api/login`, `/api/logout`, `/api/me`	Account session
`/api/photos`, `/api/photos/{id}`	Photo upload/list/get/delete

Table 5. Application page and API route map.

A.3. Content inventory snapshot

The audited non-sample data files contained the following records on 1 November 2026:

- 11 source JSON files: 3,381 collections and 33,929 problem records;
- 7 theory JSON files: 90 lessons;
- `books.json`: 216 books, magazines, and resource records;

- `docs.json`: 89 document records.

The topic assignment counts and collection breakdown are shown in Figures 11–13. These are repository measurements and should be regenerated if the content files change.

A.4. Data pipeline diagram

Figure 14 summarises how prepared content reaches the runtime pages.

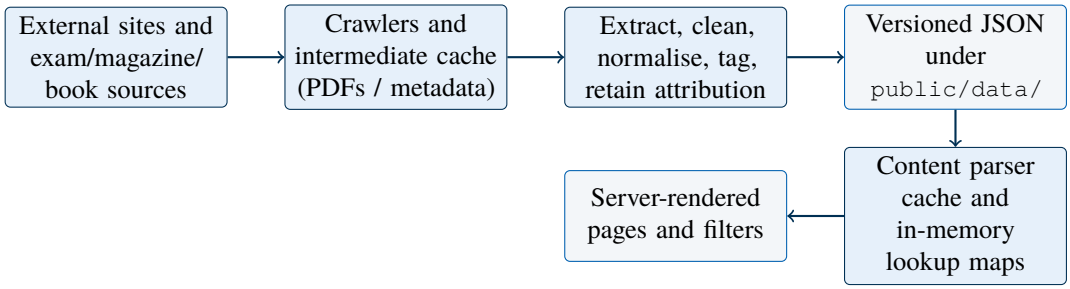


Figure 14. Content preparation and runtime consumption. The pipeline is an offline tool; it is not called during page requests. The basic version reuses its JSON output unchanged.

A.5. Glossary

Collection An ordered source-aware unit such as an exam paper or topic exercise set.

Problem One individually addressable statement nested in a collection.

Topic tag A classification label; one problem may have multiple tags.

Session cookie A signed `HttpOnly` cookie holding the user id and expiry.

Raw record count Count of JSON objects by record group before claims about uniqueness, correctness, or editorial review.

A.6. AI assistance and academic disclosure

This report draft was prepared with AI assistance (Claude) after inspecting the Rùa Toán Basic source and data files. The assistant helped organise and describe the implementation and performed the bug-fix and hardening pass described in Chapter 3; it did not run a full test suite, an accessibility audit, or an editorial review of the mathematics catalogue. The original project's content pipeline also contains exam-ingestion scripts that call Gemini and a Claude processing lane. The group should disclose any AI tools used during implementation, state which content or code they assisted with, and follow the course's academic-integrity rules.